American Terawatt

DEVELOPER INFRASTRUCTURE

Local LLM on the Shared GPU Pool

Claiming GPUs, serving an inference endpoint and answering from our own documents

End-to-end account of how a local large language model is deployed on the shared GPU pool — how GPUs are borrowed, why the work happens inside a container, where files must live to survive, how the model runtime is installed into an image without package manager access, and how the resulting endpoint is reached from a remote machine — followed by *corpus*, the retrieval system that answers questions from our own documents over that endpoint.

Revision history

REV.	DATE	AUTHOR	DESCRIPTION
0.1	Sep. 17, 2026	Eugen Starschich	Initial document
0.2	Sep. 17, 2026	Eugen Starschich	*corpus* document retrieval added (Sections 7 and 8); command reference extended
0.3	Sep. 17, 2026	Eugen Starschich	Model call sites tabulated (Section 7.4); embedding and chat endpoints distinguished

CONTENT

1 Architecture 4
1.1 Scope 4
1.2 Hosts involved 4

2 Borrowing GPUs 5
2.1 The verbs 5
2.2 What a claim actually creates 5
2.3 Why a container at all 5
2.4 Consequences to plan around 6

3 Storage: what survives 7

4 Installing the runtime 8
4.1 Staging the binary 8
4.2 Where Ollama sits 8

5 Fetching and serving the model 9
5.1 Start the server 9
5.2 Pull the weights 9
5.3 Measured behaviour 9

6 Remote access 10

7 Document retrieval on the endpoint: *corpus* 11
7.1 What it is 11
7.2 Where everything runs 11
7.3 Ingest: documents in, index out 11
7.4 Query: three model calls, not one 12

8 Operating *corpus* 14
8.1 Four ways in, one core 14
8.2 Everyday commands 14
8.3 Known limits 15
8.4 Backups 15

9 Teardown and restore 16

10 Pitfalls 17

11 Command reference 18

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

1 Architecture

1.1 Scope

This document records a working deployment of `gpt-oss:120b` under Ollama on one NVIDIA RTX PRO 6000 Blackwell card of the shared GPU pool. All concrete figures quoted in Section 5 come from that deployment.

1.2 Hosts involved

Four hosts participate. Nothing about the model runs on the developer VM; it is a control point and an SSH waypoint only.

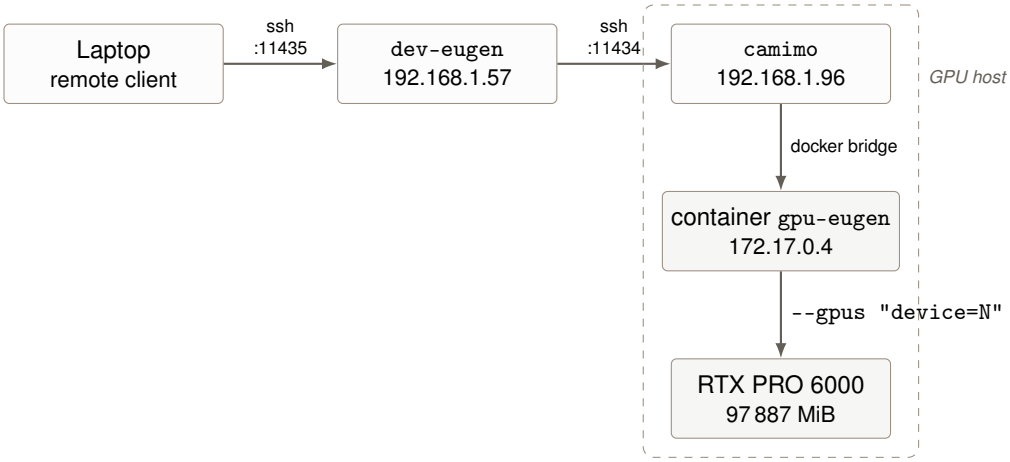


Figure 1: Hosts, container and card, with the two SSH hops that carry the endpoint out to the client.

Host	Role	Notes
Laptop	remote client	reaches the pool only via SSH
`dev-eugen`	developer VM	4 cores, 8 GB RAM, *no GPU*
`camimo`	GPU host	8 GPUs, also the NFS server for `/shared`
`gpu-eugen`	per-claim container	128 cores, 440 GB RAM visible

Table 1: The four hosts and what each contributes.

The GPU host and the storage host are the same machine: `/shared` is local disk there, and NFS-mounted on the developer VM.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

2 Borrowing GPUs

2.1 The verbs

```
gpu status        # who holds what, what is free
gpu claim 2       # take 2 (all-or-nothing: 2 or none)
gpu shell         # shell inside your container
gpu shell CMD     # run one command in it, non-interactively
gpu release       # give them back
```

`gpu` on the developer VM is a thin relay: it re-quotes its arguments and forwards them over SSH to `camimo`, where the real tool runs. Because it requests a remote TTY only when it has one, it works from scripts and from automated callers as well as interactively.

2.2 What a claim actually creates

`gpu claim` allocates cards in pool state and then starts a container pinned to exactly those cards:

```
docker run -d --name gpu-eugen \
  --gpus "device=1" \
  --label gpu.owner=eugen --label gpu.devices=1 \
  --restart no \
  --user 3003:3000 \
  -e HOME=/home/eugen \
  -v /shared:/shared \
  --shm-size 16g \
  nvidia/cuda:12.6.0-base-ubuntu24.04 sleep infinity
```

Pool state lives in `/var/lib/gpu-pool/state.json` under a `flock`, and every command reconciles that file against `docker ps` before reporting or allocating. A container that dies therefore returns its cards on its own. If the state file is unreadable the tool refuses to write rather than falling back to “the pool is empty”, which would hand out cards already in use.

2.3 Why a container at all

Property	Consequence
`--gpus "device=N"`	Isolation is structural, not by convention. Other cards are invisible; yours renumbers to 0. A stray `CUDA_VISIBLE_DEVICES` cannot reach a colleague’s run.
Container is the unit of accounting	`gpu release` is `docker rm -f`; reconciliation against `docker ps` frees the cards.
No reaper, by design	A claim is released by its owner, never by a timer — so a long training run is safe, and a forgotten claim blocks the pool.
All-or-nothing allocation	Asking for 4 with 3 free yields nothing, rather than a half-sized allocation that silently invalidates the run.
`--user uid:3000`	Files created on `/shared` are owned by you in the `team` group, not by root.

Table 2: What the container arrangement buys.

2.4 Consequences to plan around

The image is `nvidia/cuda:12.6.0-base-ubuntu24.04`. The *base* variant carries the CUDA runtime and nothing else.

Constraint	Why
No `python3`, `curl`, `git`, `tmux`	`-base` image variant
No `sudo`; `apt install` fails	runs as your uid, not root
Service ports unreachable from outside	default docker bridge, isolated netns
Filesystem is ephemeral	`gpu release` deletes the container

Table 3: Constraints imposed by the claim container.

Anything that must run inside the container therefore has to be a self-contained binary staged in from outside.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

3 Storage: what survives

Exactly one path spans both machines.

Path	Shared with container?	Survives `gpu release`?
`/shared/$USER`	yes, live and bidirectional	yes
`~` on the developer VM	**no**	n/a
`~` inside the container	it is *camimo's* home	yes, on camimo
container root filesystem	—	no

Table 4: Which paths are shared and which survive a release.

This deserves emphasis because it is easy to get wrong. The claim command bind-mounts `-v $HOME:$HOME`, but `$HOME` is resolved *on the GPU host*. The home directory visible inside the container is camimo's `/home/$USER`, not the developer VM's. A file written to `~` in one place is not visible in the other. Only `-v /shared:/shared` is unconditional, and only `/shared` genuinely spans both.

`/shared` **is not a backup.** One unmirrored NVMe copied hourly to another machine. Model weights belong there because they are large and re-downloadable; anything whose loss would matter belongs in Git, pushed elsewhere. It is also readable by the whole team, so it must never hold credentials, keys, tokens, draft commercial terms, or material under NDA.

4 Installing the runtime

4.1 Staging the binary

Because the container has no downloader and no root, the runtime is fetched on the developer VM and staged through `/shared`.

```
# on the developer VM
mkdir -p /shared/$USER/opt /shared/$USER/ollama-models
cd /shared/$USER/opt
curl -sL -o ollama.tar.zst \
  https://github.com/ollama/ollama/releases/download/v0.34.0/ollama-linux-amd64.tar.zst
mkdir -p ollama && zstd -dc ollama.tar.zst | tar -xf - -C ollama
```

This yields `bin/ollama` plus the bundled CUDA runners in `lib/ollama`. No installation step, no systemd unit, no root required — which is precisely why it works in a container where none of those are available.

4.2 Where Ollama sits

Ollama is a management layer, not the inference engine and not the model.

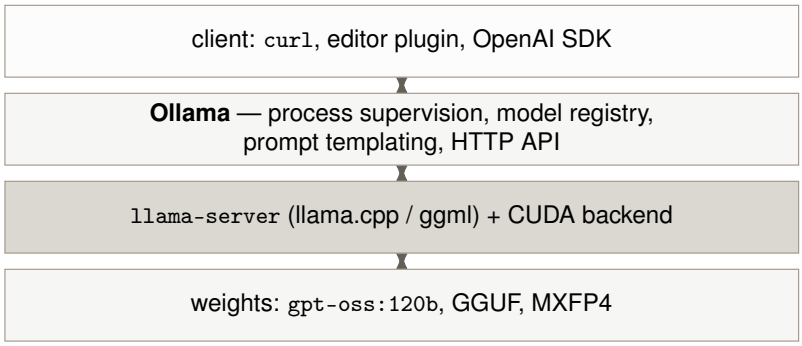


Figure 2: The serving stack, from client call down to the weights on disk.

Ollama supervises a separate `llama-server` process; llama.cpp and its CUDA kernels perform the arithmetic. Ollama contributes four things: it fetches and stores weights as blobs and manifests, it loads and unloads models with a keep-alive timer, it renders chat turns into the exact token format the model was trained on, and it exposes both a native API and an OpenAI-compatible one. It also sizes the context window to measured VRAM at start-up.

For a single user on a single card this is the right trade. For a shared, heavily concurrent endpoint, vLLM's continuous batching would serve better at the cost of a much heavier install.

5 Fetching and serving the model

5.1 Start the server

Keep this in a script on `/shared`. Long quoted one-liners do not survive the relay's re-quoting.

```
#!/bin/bash
# /shared/$USER/start-ollama.sh   --  run: gpu shell /shared/$USER/start-ollama.sh
set -u
BIN=/shared/$USER/opt/ollama/bin/ollama
LOG=/shared/$USER/ollama-serve.log

export OLLAMA_MODELS=/shared/$USER/ollama-models
export OLLAMA_HOST=0.0.0.0:11434

if ps -eo cmd | grep -q "[b]in/ollama serve"; then
    echo "already running"; exit 0
fi
cd /shared/$USER
nohup setsid "$BIN" serve > "$LOG" 2>&1 < /dev/null &
echo "launched pid $!"
```

5.2 Pull the weights

```
gpu shell bash -lc 'export OLLAMA_MODELS=/shared/$USER/ollama-models; \
  /shared/$USER/opt/ollama/bin/ollama pull gpt-oss:120b'
```

Roughly 65 GB. Because `OLLAMA_MODELS` points at `/shared`, the pull happens once and is reused by every later claim — and by teammates.

5.3 Measured behaviour

Quantity	Value
Model	`gpt-oss:120b`, 116.8 B parameters, MXFP4
Download	65 GB at ≈106–117 MB/s
Cold load from `/shared`	43 s
VRAM resident	67.0 GB of 97.9 GB
Generation, warm	183.5 tokens/s
Default context	131 072 tokens
Keep-alive	5 minutes idle, then unloaded

Table 5: Measured behaviour of `gpt-oss:120b` on one RTX PRO 6000.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

6 Remote access

The server binds inside the container's network namespace, which nothing outside can reach. Two SSH forwards carry it out. Every hop terminates on loopback, so no inference port is ever published to the office network.

```
# 1. in the container: bind the bridge interface
gpu shell /shared/$USER/start-ollama.sh

# 2. on the developer VM: forward to the container
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo

# 3. on the laptop: forward to the developer VM
ssh -N -L 11435:127.0.0.1:11434 eugen@192.168.1.57

# 4. verify, from a second terminal
curl http://localhost:11435/api/tags
```

The client then uses `http://localhost:11435/v1` as an OpenAI-compatible base URL with any non-empty API key, or `/api/...` for the native interface.

Security. Ollama performs no authentication whatsoever. Whoever reaches the port can run inference on the card and pull or delete models. Binding `0.0.0.0` inside the container exposes it to every account on the GPU host — acceptable within a team, but a deliberate decision rather than a default. Binding `0.0.0.0` on the developer VM would publish an unauthenticated endpoint to the whole LAN and should not be done. Access from outside the office requires a VPN; the developer VM has no public address.

7 Document retrieval on the endpoint: *corpus*

7.1 What it is

corpus is a retrieval-augmented question answering system over our own documents, running entirely on American Terawatt hardware and using the endpoint of Section 6 as its model. Documents are parsed once, split into passages, embedded, and indexed on local disk. At question time the relevant passages are retrieved and handed to `gpt-oss:120b` on the shared GPU pool, which answers from those passages and cites them. Nothing leaves the network, and nothing is sent to an external API.

The system deliberately does *not* fine-tune a model on the documents. Fine-tuning gives no reliable recall of specific facts, no citations, and would need re-running whenever a document changes. Retrieval at query time gives all three.

7.2 Where everything runs

Three machines are involved, and the split matters: the corpus lives next to the documents, the model lives on the GPU, and only questions and answers cross the network.

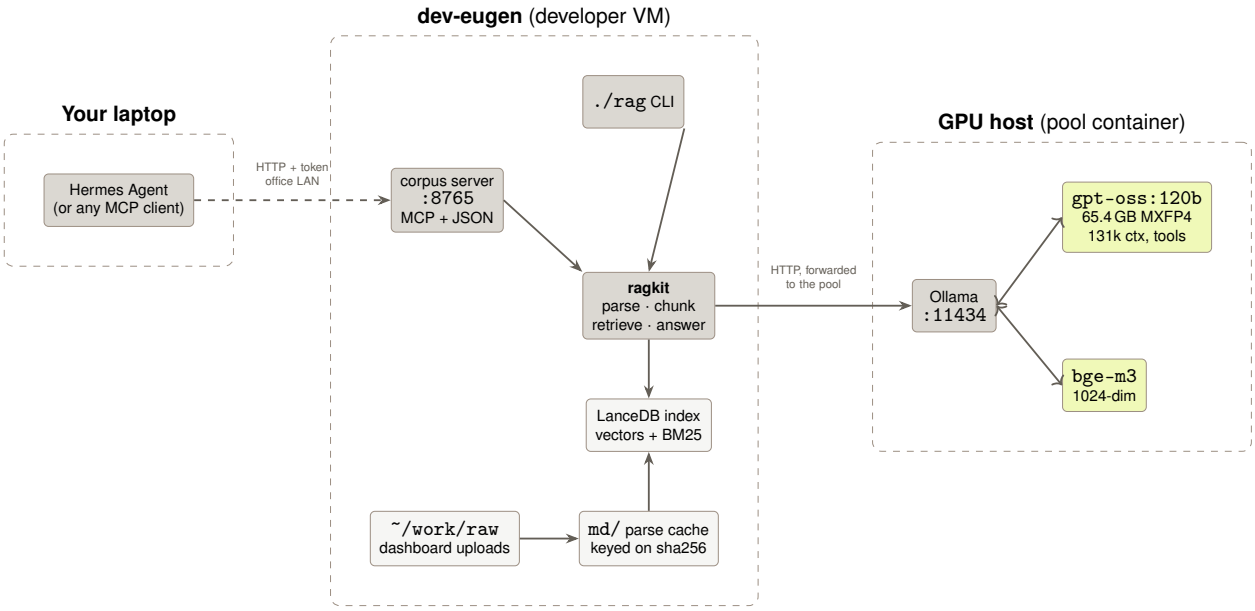


Figure 3: Where everything runs. Only questions and answers cross the network; the index stays beside the documents.

Two consequences are worth holding onto. First, **the corpus is not the model**: the laptop's SSH tunnel of Section 6 reaches the model on port 11434, but the corpus is a separate service on port 8765 and needs its own path. Second, **retrieval happens where the data is**. Only a question goes over the wire and only an answer comes back — kilobytes, not the gigabytes it would take to replicate the index.

7.3 Ingest: documents in, index out

Ingest runs once per document, and is cached. Parsing is the slow step, so its output is stored in `md/` keyed on the source file's sha256 — re-running `./rag ingest` over a growing folder only touches files whose bytes changed.

Why the chunks look the way they do

Every chunk is prefixed with its document title and heading path before it is embedded. A passage reading "the limit is 40 °C" is useless on its own; "TB 754 $>$ 6. Harmonic limits $>$ 6.11 — the limit is 40 °C" both

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

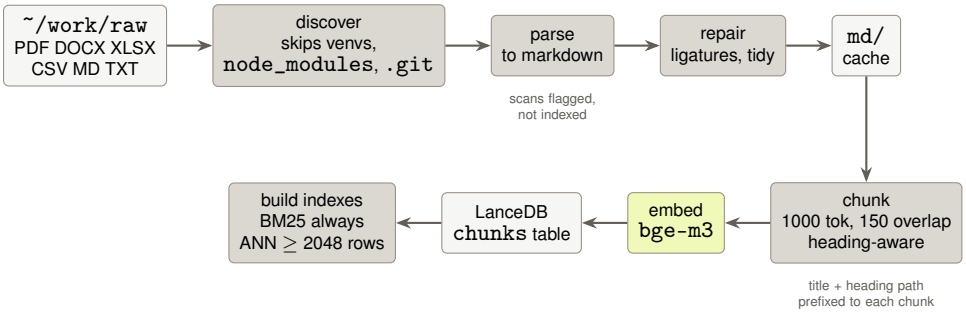


Figure 4: Ingest. Parsing is cached on the source's sha256, so re-running only touches documents whose bytes changed.

retrieves and answers correctly. The prefix is what gets embedded and BM25-indexed; the bare text is what the model is shown.

Tables are kept whole where possible, and split on row boundaries with the header repeated when they are too large — a spreadsheet row severed from its column headings cannot be read by anything.

7.4 Query: three model calls, not one

A question triggers an embedding call, then a rerank call, then the answer call. The database and the model never talk to each other; Python moves text between them.

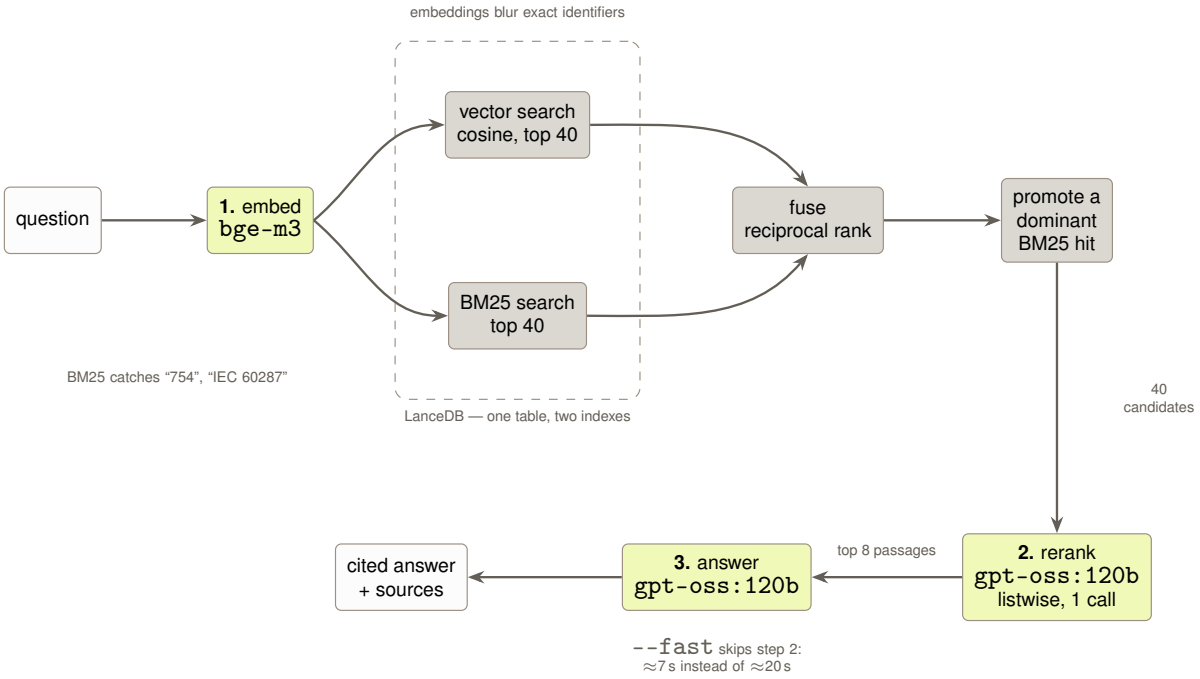


Figure 5: A query. Three model calls: embed the question, rerank the shortlist, then answer from the survivors.

Which model does what

There are exactly four model calls in the whole system, and **ragkit** makes all of them. The two models are not interchangeable: `bge-m3` only turns text into vectors — it cannot generate — so anything that produces words is `gpt-oss:120b`.

`llm.embed` posts to `/api/embed`, `llm.chat` to `/v1/chat/completions` — same Ollama, same connection, different model name in the payload. That is why Figure 3 shows one arrow into Ollama and Ollama reaching both models.

Call site	Wrapper	Model	When
`rag.py:87`	`llm.embed`	`bge-m3`	ingest: embed every chunk
`retrieve.py:62`	`llm.embed`	`bge-m3`	query: embed the question
`retrieve.py:192`	`llm.chat`	`gpt-oss:120b`	rerank the shortlist
`answer.py:36`	`llm.chat`	`gpt-oss:120b`	write the cited answer

Table 6: The four model calls, and which model serves each.

Where the LanceDB search sits in that list is: nowhere. The lookup itself (vector + BM25) uses no model at all. It is the steps on either side of it that do — embedding the question before, reranking after. Likewise "the database and the model never talk to each other" means exactly that: LanceDB and `gpt-oss:120b` have no link, and **ragkit** is the thing in the middle that talks to both.

How the model actually receives the passages

As plain text in the prompt. There is no special channel and nothing is loaded into the model. The retrieved passages are concatenated into one numbered block and sent as an ordinary chat message to `/v1/chat/completions`:

system Use only the numbered sources. Cite as [3]. If the sources do not contain the answer, say so.
user Sources:
[1] 0754 AC side harmonics > 6. Harmonic limits > 6.11.2 . . .
distortion factor of the centred interharmonic subgroup . . .
[2] . . .
Question: What harmonic order range does brochure 754 consider?

The model sees one flat sequence of tokens. It has no filesystem access, no connection to LanceDB, and no memory of previous questions — each question builds a fresh prompt. Three things follow:

- **Citations cannot drift.** The model copies back the `[n]` label it was given; the mapping from `[n]` to a file path is done in Python afterwards.
- **Refusal is honest.** "The sources do not contain any entry for brochure 9999" is an accurate report about its input.
- **Retrieval is the ceiling.** If a passage does not reach the prompt, the model cannot know it, however capable it is.

Why context size is not the constraint

	Tokens
System message	128
Retrieved passages (4) + question	4 103
Typical query at `k=8`	≈8 000
One brochure in full (TB 754)	74 550
Model context window	131 072

Table 7: Token budget of a query against the context window.

A default query uses about 6 % of the window. Stuffing the context instead would fit one brochure and a half out of roughly a thousand. That is the whole argument for retrieval: the problem was never how much text fits, it is picking the right four thousand tokens.

8 Operating *corpus*

8.1 Four ways in, one core

The rag script used in the following commands is located in:

`/home/eugen/work/corpus/rag`

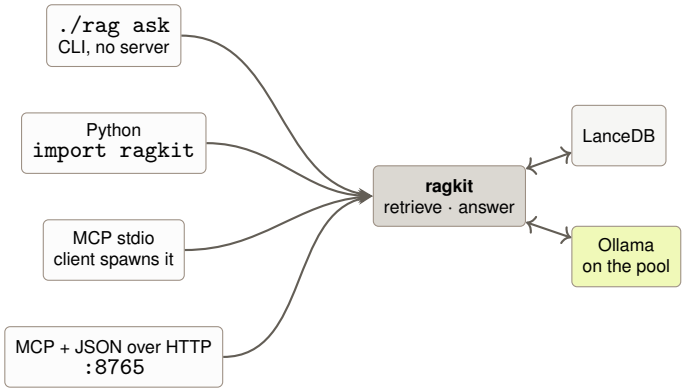


Figure 6: Four interfaces onto one pipeline — the same **ragkit** core backs every one of them.

MCP is a doorway for other programs, not part of retrieval. Working on the developer VM, it is never needed. It exists so an agent on another machine — Hermes, say — can use the corpus, and so a coding session can search the documents mid-conversation.

Interface	Server?	Auth	Use it for
`./rag ask`	no	—	everyday use on the developer VM
`import ragkit`	no	—	notebooks, own scripts
MCP stdio	spawned	—	Claude Code on that box
MCP `/mcp`	yes	bearer	Hermes and other agents
JSON `/ask`, `/search`	yes	bearer	curl, scripts, any language

Table 8: The four interfaces and what each is for.

8.2 Everyday commands

Task	Command
Index new uploads	`./rag ingest`
Index elsewhere	`./rag ingest /shared/eugen/brochures`
Re-parse everything	`./rag ingest --force`
Rebuild from scratch	`./rag ingest --reset`
Ask	`./rag ask "..."`
Passages only	`./rag search "..."`
What is indexed	`./rag status`
Start the server	`tmux new -s corpus-mcp`, then `./mcp-http --host 0.0.0.0`
Client details	`./mcp-http --show-client-config`

Table 9: Everyday *corpus* commands on the developer VM.

Documents reach the developer VM through the **dashboard upload panel** at

`http://dev.americanterawatt.com/`

landing in `~/work/raw`. That is the route; there is no SSH access for anyone, by design.

8.3 Known limits

- **Scanned PDFs index nothing.** Ingest flags them rather than silently adding empty chunks. A vision model (`qwen2.5vl:32b`) is now on the pool and is the obvious OCR path if needed.
- **Damage already in the source is not repaired.** The eCIGRE brochure list contains titles that lost their ligatures before reaching the spreadsheet (“Model specication”), so a search for “specification” will not match them. Ligature repair runs only for PDFs, where the document’s own text supplies a trustworthy vocabulary.
- **One writer.** LanceDB is embedded: concurrent readers are fine, two simultaneous ingests are not.
- **Rerank latency.** The 120B spends reasoning tokens before any visible output, so a reranked query costs ≈20 s.
- **LAN exposure.** With `--host 0.0.0.0` the bearer token is the only thing between the office network and every indexed document.

8.4 Backups

`index/` and `md/` are derived — they can be deleted and rebuilt from the sources at any time. The pipeline code is what matters, and as Section 3 notes `/shared` is one unmirrored NVMe, so that directory belongs in Git, pushed elsewhere.

9 Teardown and restore

```
gpu release        # deletes the container; cards return to the pool
```

The container, the running server and both tunnels are destroyed. The Ollama installation, the model weights and the start-up script all live on `/shared` and survive. Restoring service after a later claim costs two commands and no re-download:

```
gpu claim 1
gpu shell /shared/$USER/start-ollama.sh
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo
```

Confirm the bridge address first, as it may differ between claims:

```
ssh camimo "docker inspect -f \
  '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' gpu-eugen"
```

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

10 Pitfalls

Symptom	Cause and remedy
Claim never released	Claims do not expire, by design. A forgotten claim blocks the pool indefinitely. Release explicitly.
`Text file busy` on a freshly staged binary	NFS write-back lag on a new executable under `/shared`. It clears by itself; wait rather than re-copying.
`pgrep -f "ollama serve"` always matches	The search pattern appears in the searching command's own command line. Use `grep "[b]in/ollama serve"`.
Quoted one-liner silently does nothing	The relay re-quotes arguments; nested quoting does not survive. Put the commands in a script on `/shared`.
`curl` returns the wrong model	A local Ollama already occupies port 11434, so the forward failed to bind and the query answered locally. Forward to a free local port such as 11435.
`ssh -N` appears to hang	Expected. `-N` runs no remote command; it holds the tunnel open and prints nothing. Test from a second terminal.
Work lost on release	Only `/shared` spans both machines. The container home is the GPU host's, not the developer VM's.
`<defunct>` processes accumulate	Container PID 1 is `sleep infinity` and never reaps children. Harmless; cleared on release.

Table 10: Symptoms seen in practice, with cause and remedy.

11 Command reference

```
# --- pool ------------------------------------------------------------
gpu status
gpu claim 1
gpu shell                       # interactive
gpu shell /path/to/script.sh    # one command
gpu release

# --- inside the container --------------------------------------------
nvidia-smi
ps -eo pid,etime,cmd | grep "[b]in/ollama serve"
tail -f /shared/$USER/ollama-serve.log

# --- model -----------------------------------------------------------
ollama list
ollama pull  gpt-oss:120b
ollama run   gpt-oss:120b --verbose "prompt"
ollama stop  gpt-oss:120b       # unload from VRAM, keep on disk

# --- endpoint --------------------------------------------------------
curl http://localhost:11435/api/tags
curl http://localhost:11435/api/generate \
  -d '{"model":"gpt-oss:120b","prompt":"...","stream":false}'
```

corpus, on the developer VM

```
# --- index -----------------------------------------------------------
./rag ingest                            # new uploads in ~/work/raw
./rag ingest /shared/eugen/brochures    # a folder elsewhere
./rag ingest --force                    # re-parse everything
./rag ingest --reset                    # rebuild from scratch
./rag status                            # what is indexed

# --- ask -------------------------------------------------------------
./rag ask    "..."                      # cited answer
./rag ask    "..." --fast               # skip the rerank call
./rag search "..."                      # passages only

# --- server ----------------------------------------------------------
tmux new -s corpus-mcp
./mcp-http --host 0.0.0.0               # MCP + JSON on :8765, bearer token
./mcp-http --show-client-config
```

AMERICA'S PRIVATE INDUSTRIAL POWER GRID